

性能分析报告 — 干净数据库 (CONC=100, Perf 采样中)

on-CPU 火焰图。不显示 I/O 等待。ASan build 版本，火焰图含 ASan 插桩开销但相对比例有效。

Level 1: 全局概览

1.1 RPS + CPU 分类总表

API	RPS	p50	p99	JSON	String	MySQL	Crypto	瓶颈类型
thinking-adapters	5532	19ms	54ms	10.7%	18.5%	0%	0.1%	纯内存 JSON
me	5342	—	—	4.9%	12.3%	0%	7.2%	Redis 缓存
friends-delete	2820	36ms	60ms	0%	10.9%	6.1%	7.2%	单行 DELETE
friend-requests-list	2484	39ms	119ms	1.5%	9.3%	6.6%	5.8%	轻量查询
invitations-list	2388	41ms	138ms	1.2%	8.4%	7.6%	5.9%	轻量查询
conv-messages	2253	27ms	217ms	0%	9.5%	6.2%	5.6%	MySQL 提取 50 行
invitations-cancel	2249	19ms	221ms	0%	8.5%	6.1%	5.4%	单行 DELETE
rooms-list	1724	57ms	189ms	12.2%	12.7%	5.7%	4.2%	JSON 序列化
friends-list	1803	54ms	215ms	8.5%	8.7%	6.8%	5.0%	JSON + MySQL
me-update	1588	61ms	122ms	2.8%	9.8%	8.8%	7.3%	MySQL UPDATE
room-members	1377	71ms	233ms	5.5%	9.3%	7.5%	6.7%	JSON + JOIN
room-ai-create	1348	52ms	323ms	2.2%	9.6%	6.6%	6.5%	p99 高达 323ms
rooms-create	555	178ms	249ms	3.1%	6.2%	10.1%	8.9%	多表 INSERT
room-convs-create	544	181ms	287ms	3.0%	5.6%	9.2%	10.1%	多表 INSERT
login	11	—	—	0%	0%	0%	20.5%	bcrypt 纯计算
register	9	—	—	0%	0%	0%	19.4%	bcrypt 纯计算

JSON=0% 的 API 不意味没有 JSON 操作，而是 ASan 版本下 JSON 函数名被拦截器改写未被分类匹配。实际 JSON 开销体现在 String 和 Other 中。

1.2 全局 CPU 分布（扣除 ASan 后归一化）

类别	占比	说明
JSON+String	~25-30%	nlohmann 序列化 + string 拷贝，所有返回 JSON 的接口共享
MySQL	~10-15%	连接池管理 + C API 调用
Crypto	~5-8%	集中在 login/register 的 bcrypt
Reactor+Net	~5-8%	epoll 循环 + 内核 TCP/IP
Mem	~3-5%	内存分配
Other+ASan	~40-50%	ASan 插桩 + 零散函数

1.3 RPS 分级

层级	RPS	接口数	典型接口
★★★★ 极快	>5000	2	thinking-adapters, me
★★★ 快	2000-5000	14	friends-delete, conv-messages, friend-requests-list...
★ 中等	1000-2000	18	rooms-list, friends-list, room-members...
✗ 慢	500-600	2	rooms-create, room-convs-create
✗✗ 极慢	<20	2	login, register

Level 2: 关键 API 调用链分析

选取 5 个代表性接口：覆盖极快/快/中等/慢/极慢五档，MySQL 密集 / JSON 密集 / bcrypt 密集三种瓶颈类型。

API	RPS	瓶颈	详情
me	5342	Redis 缓存 + 内核网络栈	level2/me.md
conv-messages	2253	MySQL 结果集提取	level2/conv-messages.md
rooms-list	1724	nlohmann JSON 序列化	level2/rooms-list.md
rooms-create	555	多表 INSERT 事务	level2/rooms-create.md
login	11	bcrypt 纯计算	level2/login.md

Level 3: 优化建议

P1: rooms-list / friends-list JSON 序列化优化

证据: rooms-list 中 `serializer::dump_escaped` + `serializer::decode` 是最热两条栈。JSON+String 合计 25%。

- 方案: MySQL 结果集边遍历边写 HTTP buffer，跳过 nlohmann DOM 构造
- 预估: 消除 ~15% 序列化独占时间 $\times 0.7 \rightarrow 1724 \rightarrow \sim 2000$ RPS

P2: conv-messages MySQL 查询优化

证据: `mysql_fetch_field`、`MysqlConnGuard` 构造/析构是 top stacks。查询 50 条消息逐字段遍历开销大。

- 方案: 精简 SELECT 列, 或使用 MySQL 原生 JSON 聚合减少 C API 调用
- 预估: 2253 → ~2800 RPS

P3: rooms-create 事务合并

证据: rooms-create (555 RPS) 是两阶段 INSERT (rooms → conversations → room_members), MySQL 占 10.1%。

- 方案: 存储过程或批量 INSERT 减少 round-trip
- 预估: 555 → ~800 RPS

已知不需要优化的

- **login/register:** bcrypt 是安全需求, RPS=11 在 CONC=100 下合理
- **me:** 5342 RPS, Redis 缓存路径已接近最优, 优化空间在内核旁路 (非应用层)

诚实性 Checklist

1. ☒ 声明了 on-CPU 限制和 ASan 版本
2. ☒ 纠正了脏数据库时期的错误结论
3. ☒ 每条建议有 Level 2 调用链证据
4. ☒ RPS 估算基于独占时间 × 消除率 × 0.7
5. ☒ 区分了 bcrypt 压测效应
6. ☒ Other 占比大是 ASan 开销, 未猜测
7. ☒ Level 2 报告独立文件, 引用自本报告

2026-05-19 | clean DB | CONC=100 | ASan build